Bölüm 2 Temel Programlama

Bir Örnekle Programlamaya Giriş

Bir Alanın Hesaplanması

Daire

Bu program, alanı hesaplar

dairenin.

ComputeArea

Koş

animasyon

Bir Programın Çalışmasını İzleme

```
public class AlanHesapla {
  /** Ana yöntem */
  public static void main(String[] args) {
    double yarıçap;
    double alan;

    // Bir yarıçap atama
    yarıçap = 20;

    // Alanı hesapla
    alan = yarıçap * yarıçap * 3.14159;

    // Sonuçları göster
    System.out.println("Yarıçapı için dairenin alanı " +
      radius + " şu " + area);
  }
}
```

bellek ayırmak

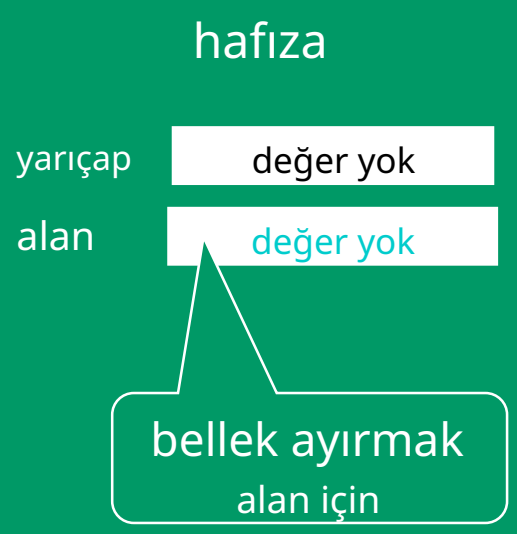
yarıçap için

yarıçap | no value

animasyon

Bir Programın Çalışmasını İzleme

```
public class AlanHesapla {
  /** Ana yöntem */
  public static void main(String[] args) {
    double yarıçap;
    double area;

    // Bir yarıçap atama
    radius = 20;

    // Alanı hesapla
    area = radius * radius * 3.14159;

    // Sonuçları göster
    System.out.println("Yarıçapı için dairenin alanı " +
      radius + " şu " + area);
  }
}
```



animasyon

Bir Programın Çalışmasını İzleme

```
public class AlanHesapla {
  /** Ana yöntem */
  public static void main(String[] args) {
    double yarıçap;
    double alan;

    // Bir yarıçap atama
    yarıçap = 20;

    // Alanı hesapla
    alan = yarıçap * yarıçap * 3.14159;

    // Sonuçları göster
    System.out.println("Yarıçapı için dairenin alanı " +
      radius + " şu " + area);
  }
}
```

animasyon

Bir Programın Çalışmasını İzleme

```
public class AlanHesapla {
  /** Ana yöntem */
  public static void main(String[] args) {
    double yarıçap;
    double alan;

    // Bir yarıçap atama
    yarıçap = 20;

    // Alanı hesapla
    area = radius * radius * 3.14159;

    // Sonuçları göster
    System.out.println("Yarıçapı için dairenin alanı " +
      radius + " şu " + area);
  }
}
```

hafıza

yarıçap	20
alan	1256.636

alanı hesapla ve onu

değişken alanına ata

animasyon

Bir Programın Çalışmasını İzleme

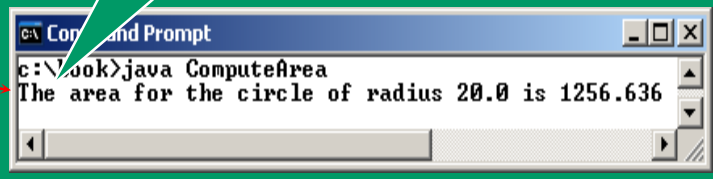
```
public class AlanHesapla {
  /** Ana yöntem */
  public static void main(String[] args) {
    double yarıçap;
    double alan;

    // Bir yarıçap atama
    yarıçap = 20;

    // Alanı hesapla
    alan = yarıçap * yarıçap * 3.14159;

    // Sonuçları göster
    System.out.println("Yarıçapı için dairenin alanı " +
      radius + " is " + area);
  }
}
```

hafıza

yarıçap	20
alan	1256.636

bir mesajı

konsola yazdır



Konsoldan Girdi Okuma

1. Bir Scanner nesnesi oluştur

```
Scanner input = new Scanner(System.in);
```

2. next(), nextByte(), nextShort(), nextInt(), nextLong(), nextFloat(), nextDouble() veya nextBoolean() yöntemlerini kullanarak bir dize, byte, short, int, long, float, double veya boolean değeri. Örneğin,

```
System.out.print("Bir double değeri girin: ");
Scanner input = new Scanner(System.in);
double d = input.nextDouble();
```

ComputeAreaWithConsoleInput

Koş

ComputeAverage

Koş

Tanımlayıcılar

- Bir tanımlayıcı,

  harfler, rakamlar, alt çizgiler (_) ve dolar işaretlerinden ($) oluşan bir karakter dizisidir.

- Bir tanımlayıcı bir harf, bir alt çizgi (_) veya bir dolar işareti ($) ile başlamalıdır.

  Rakamla başlayamaz.

  - Bir tanımlayıcı, rezerve edilmiş bir kelime olamaz.

- Bir tanımlayıcı `true`, `false` veya

  `null` olamaz.

- Bir tanımlayıcı herhangi bir uzunlukta olabilir.

Değişkenler

```
// İlk alanı hesapla
yarıçap = 1.0;
alan = yarıçap * yarıçap * 3.14159;
System.out.println("Alan “ +
   alan + " yarıçap için "+yarıçap);

// İkinci alanı hesapla
radius = 2.0;
alan = yarıçap * yarıçap * 3.14159;
System.out.println("Alan “ +
   alan + " yarıçap için "+yarıçap);
```

Değişkenleri Tanımlama

```
int x;          // x'i bir
                // tam sayı değişkeni olarak tanımla;
double radius; // radius'u
                // bir double değişkeni olarak tanımla;
char a;         // a'yı bir
                // karakter değişkeni olarak tanımla;
```

Atama İfadeleri

```
x = 1;          // x'e 1 ata;

radius = 1.0;   // radius'a 1.0 ata;

a = 'A';        // a'ya 'A' ata;
```

Bir Adımda Bildirme ve Başlatma

- `int x = 1;`
- `double d = 1.4;`

Sabitler

final veri_türü SABİT_ADI = DEĞER;

```
final double PI = 3.14159;
```

final int BOYUT = 3;

Sayısal Veri Türleri

Ad	Aralık	Depolama Boyutu
bayt	–27 (-128) ile 27–1 (127)	8-bit işaretli
kısa	–215 (-32768) ile 215–1 (32767)	16-bit işaretli
tam sayı	–231 (-2147483648) ile 231–1 (2147483647)	32-bit işaretli
uzun	–263 to 263–1 (yani, -9223372036854775808 tan 9223372036854775807)	64-bit signed
float	Negatif aralık: -3.4028235E+38 ile -1.4E-45 Pozitif aralık: 1.4E-45 ile 3.4028235E+38	32-bit IEEE 754
double	Negatif aralık: -1.7976931348623157E+308 to -4.9E-324 Pozitif aralık: 4.9E-324 ile 1.7976931348623157E+308	64-bit IEEE 754

Sayısal Operatörler

Ad	Anlam	Örnek	Sonuç
+	Ekleme	34 + 1	35
-	Çıkarma	34.0 – 0.1	33.9
*	Çarpma	300 * 30	9000
/	Bölme	1.0 / 2.0	0.5
%	Kalan	20 % 3	2

Tam Bölme

+, -, *, / ve %

5 / 2, bir tam sayı 2 verir.

5.0 / 2, bir double değeri 2.5 verir.

5 % 2, 1 verir (bölmenin kalanı)

Kalan Operatörü

Kalan, programlamada çok faydalıdır. Örneğin, bir çift sayı % 2 her zaman 0 ve tek sayı % 2 her zaman 1. Bu özelliği, bir sayının olup olmadığını belirlemek için kullanabilirsiniz. bugün Cumartesi olduğunu ve senin ve senin Arkadaşlar 10 gün içinde buluşacak. 10 gün sonra hangi gün? Aşağıdaki bilgiyi kullanarak o günün Salı olduğunu bulabilirsiniz. ifade:

Cumartesi, haftanın 6ıncı günüdür

Bir haftada 7 gün vardır

(6 + 10) % 7 2'dir

Bir haftadaki 2ci gün Salı'dır

10 gün sonra

Problem: Zamanı Gösterme

Bir program yazın, saniyelerden saat ve dakikaları elde etsin.

DisplayTime

Koş

NOT

Kesirli sayılarla ilgili hesaplamalar

yaklaşık olarak yapılmaktadır çünkü bu sayılar saklanmamaktadır.

tam doğrulukla. Örneğin,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

0.0000000000000000 gösterir, 0.5 değil, ve

```
System.out.println(1.0 - 0.9);
```

0.0999999999999998'i gösterir, 0.1 değil. Tam sayılar

tam olarak saklanır. Bu nedenle, tam sayılarla yapılan hesaplamalar

tam bir sonuç verir.

Sayısal Literaller

Bir *literal*, programda doğrudan görünen bir sabit değerdir.
Örneğin, 34, 1.000.000 ve
5.0, aşağıdaki ifadelerde literallerdir:

```
int i = 34;

long x = 1000000;

double d = 5.0;
```

Tamsayı Literalleri

Bir tamsayı literal, değişkenin içine sığabildiği sürece bir tamsayı değişkenine atanabilir.

Eğer literal, değişkenin tutabileceğinden çok büyükse, bir derleme hatası oluşur.

Örneğin, byte b = 1000 ifadesi

1000'in saklanamayacağı için bir derleme hatası oluşturur byte türünde bir değişkende.

Bir tam sayı literali, int türünde olduğu varsayılır, bunun değerler-2^{31}(-2147483648) ile 2^{31}–1 arasında (2147483647). Uzun türde bir tam sayı literalini belirtmek için, L veya l harfi ile ekleyin. L tercih edilir çünkü l (küçük l) kolayca 1 (rakam) ile karıştırılabilir bir).

Kesirli Sayı Literalleri

Kesirli sayılar ondalık
nokta ile yazılır. Varsayılan olarak, bir kesirli sayı çift türü değer olarak kabul edilir. Örneğin, 5.0 bir çift değer olarak kabul edilir, bir float değeri olarak değil. Siz bir sayıyı float yapmak için sonuna f harfi ekleyebilirsiniz ve bir sayıyı çift yapmak için ekleyin letter d or D. For example, you can use 100.2f or 100.2F bir ondalık sayı için, ve 100.2d veya 100.2D çift sayı için.

Bilimsel Notasyon

Kesirli sayılar da belirtilebilir bilimsel gösterim, örneğin, 1.23456e+2, 1.23456e2 ile aynı olup, 123.456'ya eşittir, ve 1.23456e-2, 0.0123456'ya eşittir. E (veya e) bir üssü temsil eder ve bu ya

küçük harf veya büyük harf.

Aritmetik İfadeler

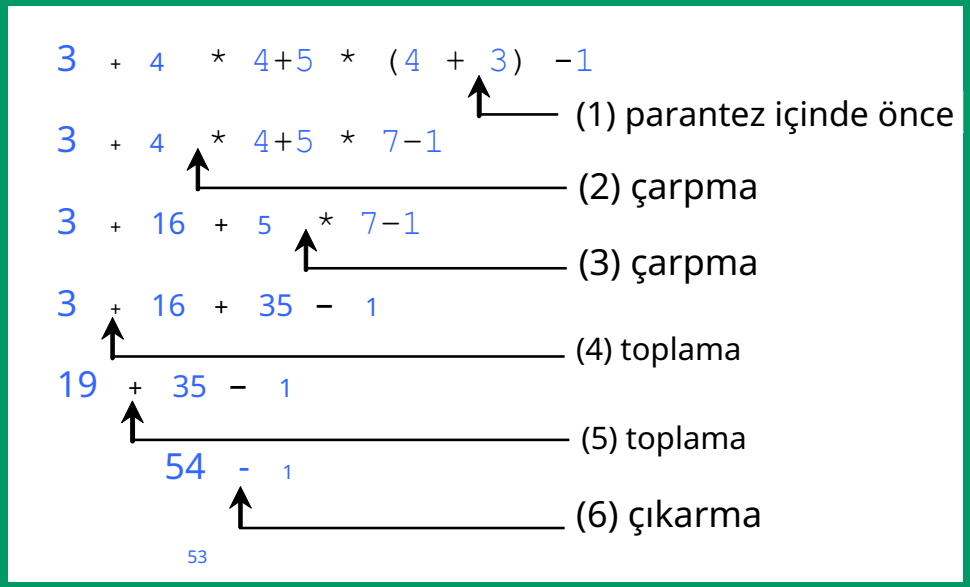
$$\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)$$

şu şekilde çevrilir

(3+4*x)/5 – 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)

Bir İfadenin Değerlendirilmesi

Java'nın bir değerlendirme yöntemi vardır
Java'nın arka planındaki ifade
ifade ve ona karşılık gelen aritmetik ifade
aynıdır. Bu nedenle, güvenle uygulayabilirsiniz
Java ifadesini değerlendirmek için aritmetik kural.



Problem: Sıcaklıkları Dönüştürme

Bir Fahrenheit derecesini

Celsius'a dönüştüren bir program yazın, formül kullanarak:

$$celsius = (\tfrac{5}{9})(fahrenheit - 32)$$

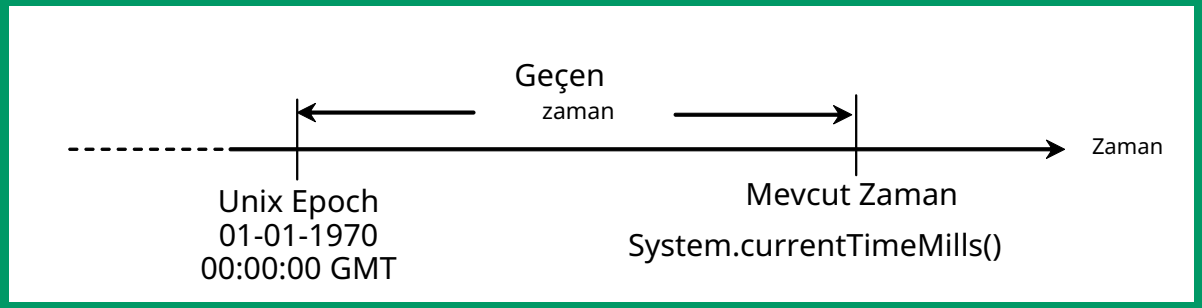
FahrenheitToCelsius

Koş

Mevcut Zamanı Gösterme Problemi

GMT'de mevcut zamanı gösteren bir program yazın format saat:dakika:saniye şeklinde 1:45:19 gibi.

System sınıfındaki currentTimeMillis metodu döner 1 Ocak'tan itibaren milisaniye cinsinden mevcut zaman 1, 1970 GMT. (1970, Unix işletim sisteminin resmi olarak tanıtıldığı yıl.) Bu yöntemi kullanabilirsiniz mevcut zamanı almak ve ardından mevcut saniye, dakika ve saati aşağıdaki gibi hesaplamak.



ShowCurrentTime

Koş

Kısayol Atama Operatörleri

Operatör	*Örnek*	*Eşdeğer*
`+=`	`i += 8`	`i = i + 8`
`-=`	`f -= 8.0`	`f = f - 8.0`
`*=`	`i *= 8`	`i = i * 8`
`/=`	`i /= 8`	`i = i / 8`
`%=`	`i %= 8`	`i = i % 8`

Artırım ve Azaltma Operatörleri

Operatör	İsim	Açıklama
++değişken	ön artış	İfade (++değişken), değişkeni 1 artırır ve artıştan sonra değişkendeki yeni değeri değerlendirir.
değişken++	sonra artış	İfade (değişken++) orijinal değeri değerlendirir değişkende ve değişkeni 1 artırır.
--değişken	ön azaltma	İfade (--değişken), değişkeni 1 azaltır ve değerlendirir değişimden sonra var'daki yeni değere.
var--	postdecrement	İfade (var--) *orijinal* değeri değerlendirir var içinde ve var'ı 1 azaltır.

Artırma ve
Azaltma Operatörleri, devam.

```
int i = 10;
int newNum = 10 * i++;
```

Aynı etki →

```
int newNum = 10 * i;
i = i + 1;
```

```
int i = 10;
int newNum = 10 * (++i);
```

Aynı etki →

```
i = i + 1;
int newNum = 10 * i;
```

Sayısal Tür Dönüşümü

Aşağıdaki ifadeleri dikkate alın:

```
byte i = 100;
long k = i * 3 + 4;
double d = i * 3.1 + k / 2;
```

Dönüşüm Kuralları

İki

farklı türdeki operandı içeren bir ikili işlem gerçekleştirirken, Java otomatik olarak

operandı aşağıdaki kurallara göre dönüştürür:

1. Eğer operandlardan biri double ise, diğeri double'a dönüştürülür.
2. Aksi takdirde, eğer operandlardan biri float ise, diğeri float'a dönüştürülür.
3. Aksi takdirde, eğer operandlardan biri long ise, diğeri long'a dönüştürülür.
4. Aksi takdirde, her iki operand da int'e dönüştürülür.

Tür Dönüştürme

İçsel Dönüştürme

double d=3;(tip genişletme)

Açık Dönüştürme

int i=(int)3.0;(tip daraltma)

int i = (int)3.9; (Kesir kısmı kesilmiş)

aralık artar

byte, short, int, long, float, double

Karakter Veri Türü

Dört onaltılık basamak.

char letter = 'A'; (ASCII)

char numChar = '4'; (ASCII)

char letter = '\u0041'; (Unicode)

char numChar = '\u0034'; (Unicode)

NOT: Artırma ve azaltma operatörleri de kullanılabilir

Unicode karakterini almak için char değişkenleri üzerinde bir sonraki veya önceki karakteri elde etmek.

Örneğin, aşağıdaki ifadeler karakter b'yi gösterir.

```
char ch = 'a';
System.out.println(++ch);
```

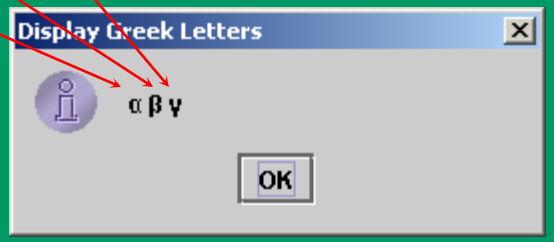
Unicode Format

Java karakterleri *Unicode* kullanır, 16 bitlik bir kodlama şemasıdır.

Unicode Konsorsiyumu tarafından desteklenmek üzere kurulmuştur

yazılı metinlerin değişimi, işlenmesi ve görüntülenmesi dünyanın çeşitli dilleri. Unicode iki bayt alır, önden gelen \u, dört onaltılık sayı ile ifade edilir '\u0000' ile '\uFFFF' arasında değişen karakterleri temsil edebilir 65535 + 1 karakter.

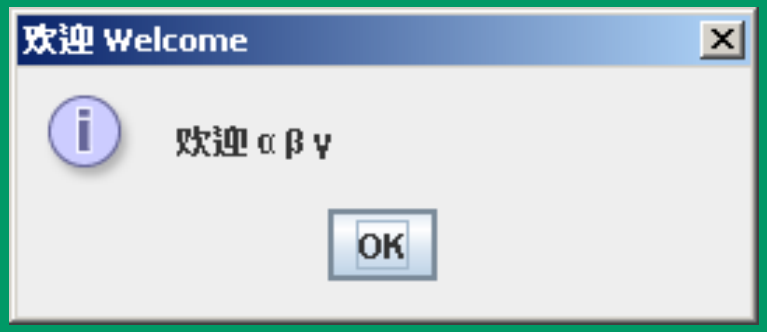
Unicode \u03b1 \u03b2 \u03b3 üç Yunan harfi için



Problem: Unicode'ları Gösterme

İki Çince

karakter ve üç Yunan harfi gösteren bir program yazın.



GösterUnicode

Koş

Özel Karakterler için Kaçış Dizileri

Açıklama	*Kaçış Dizisi*	*Unicode*
Geri Tuşu	`\b`	`\u0008`
Sekme	`\t`	`\u0009`
Satır Beslemesi	`\n`	
Carriage return	`\r`	
Backslash	`\\`	`\u005C`
Tek Tırnak	`'`	`'`
Double Quote	`\"`	`\"`

Karakter ve

Sayısal Türler Arasında Dönüşüm

`int i = 'a';` // int i = (int)`'a'`; ile aynıdır;

char c = 97; // char c = (char)97; ile aynıdır;

String Türü

char türü yalnızca bir karakteri temsil eder. Bir karakterler dizisini temsil etmek için String adı verilen veri türünü kullanın.

Örneğin,

String message = "Java'ya Hoş Geldiniz";

String, aslında Java kütüphanesinde önceden tanımlanmış bir sınıftır, tıpkı System sınıfı ve JOptionPane sınıfı. String türü bir ilkel tür değildir. Referans türü olarak bilinir. Herhangi bir Java sınıfı bir değişken için referans türü olarak kullanılabilir. Referans veri türleri 7. Bölüm “Nesneler ve Sınıflar”da kapsamlı bir şekilde tartışılacaktır. Şimdilik, bir String değişkenini nasıl tanımlayacağınızı, değişkene bir dize atamayı ve dizeleri nasıl birleştireceğinizi bilmeniz yeterlidir.

Dize Birleştirme

```
// Üç dize birleştiriliyor
String message = "Hoş geldiniz " + "Java'ya " + "giriş";

// Dize Chapter sayısı 2 ile birleştiriliyor
String s = "Bölüm" +  2; // s, Bölüm2 oluyor

// Dize Supplement karakter B ile birleştiriliyor
String s1  = "Ek" + 'B'; // s1, EkB oluyor
```

Programlama Tarzı ve Dokümantasyon

- Uygun Yorumlar
- İsimlendirme Kuralları
- ☞ DoğruGirintiVeBoşluk
  - Satırlar
- Blok Stilleri

Uygun Yorumlar

Başında bir özet ekleyin
programın ne yaptığını, anahtarını açıklamak için
özellikleri, destekleyici veri yapıları ve kullandığı herhangi bir
benzersiz teknik.

Adınızı, sınıf bölümünüzü, eğitmeninizi, tarihi ekleyin,
ve programın başında kısa bir açıklam
a.

İsimlendirme Kuralları

- ☞ Anlamlı ve açıklayıcı isimler seçin.

Değişkenler ve yöntem adları:

- küçük harf kullanın. Eğer isim birden fazla kelimeden oluşuyorsa, hepsini birleştirip küçük harf kullanın ilk kelime için, ilk harfi büyük yapın isimdeki her bir sonraki kelimenin. Ö rneğin, değişkenler yarıçap ve alan, ve alanı_hesapla.

İsimlendirme Kuralları, devam.

- Sınıf adları:
  - Her Kelimenin İlk Harfini Büyütün isim. Örneğin, sınıf adı `ComputeArea`.

- Sabitler:
  - Sabitlerdeki tüm harfleri büyük yapın ve kullanın kelimeleri bağlamak için alt çizgiler. İçin örnek, sabit PI ve MAX_VALUE

Uygun Girinti ve Boşluk

- Girinti
  - İki boşluk girintisi yapın.
- Boşluk
  - Kodun bölümlerini ayırmak için boş satır kullanın.

Blok Stilleri

Küme parantezleri için satır sonu stilini kullanın.

Sonraki satır stili

```
public class Test
{
    public static void main(String[] args)
    {
        System.out.println("Block Styles");
    }
}
```

Satır sonu stili

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Block Styles");
    }
}
```

Programlama Hataları

- Söz Dizimi Hataları
  - Derleyici tarafından tespit edildi
- Çalışma Zamanı Hataları
  - Programın iptal edilmesine neden olur
- Mantık Hataları
  - Yanlış sonuç üretir

Sözdizimi Hataları

```
public class ShowSyntaxErrors {
  public static void main(String[] args) {
    i = 30;
    System.out.println(i + 4);
  }
}
```

Çalışma Zamanı Hataları

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    int i = 1 / 0;
  }
}
```

Mantık Hataları

```
import java.util.Scanner;// Scanner java.util içinde
public class ShowLogicErrors {
    // Bir sayının 1 ile 100 arasında (dahil) olup olmadığını belirleyin
    public static void main(String[] args) {
        // Kullanıcıdan bir sayı girmesini isteyin
        Scanner input = new Scanner(System.in);
        System.out.print("Bir tam sayı girin: ");
        int number = input.nextInt();

        // Sonucu göster
        System.out.println("The numara dır arasında 1 ve 100
    inclusively? " + ((1 > number) && (number > 100)));
        System.exit(0);
    }
}
```

Hata ayıklama

Mantık hatalarına *hata* denir. Hata bulma ve hataları düzeltme sürecine hata ayıklama denir. Hata ayıklamanın yaygın bir yaklaşımı bir dizi yöntemi birleştirerek hatanın bulunduğu programın kısmına daraltmaktır.

Programı elle izleyebilirsiniz (yani, hataları programı okuyarak yakalayabilirsiniz) veya değişkenlerin değerlerini veya programın yürütme akışını göstermek için print ifadeleri ekleyebilirsiniz.

Bu yaklaşım kısa, basit bir program için işe yarayabilir. Ancak büyük, karmaşık bir program için, hata ayıklamanın en etkili yaklaşımı bir hata ayıklayıcı kullanmaktır. yararlı araçtır.

Hata Ayıklayıcı

Hata ayıklayıcı, hata ayıklamayı kolaylaştıran bir programdır.

Bir hata ayıklayıcı kullanarak

- Bir ifadeyi tek seferde çalıştırabilirsiniz.
- Bir yönteme adım atabilir veya geçebilirsiniz.
- Kesme noktaları ayarlayabilirsiniz.
- Değişkenleri görüntüleyebilirsiniz.
- Çağrı yığınını görüntüleyebilirsiniz.
- Değişkenleri değiştirebilirsiniz.

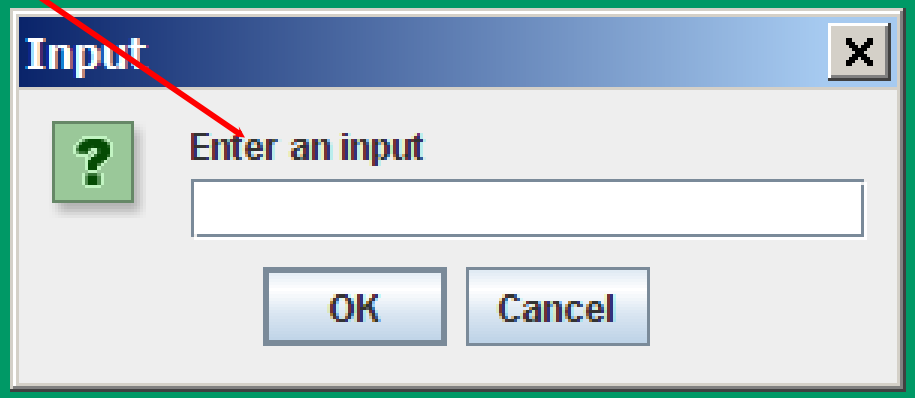
JOptionPane Girişi

Bu kitap, girdi almanın iki yolunu sunmaktadır.

1. Scanner sınıfını kullanarak (konsol girişi)
2. JOptionPane girdi diyalog kutularını kullanarak

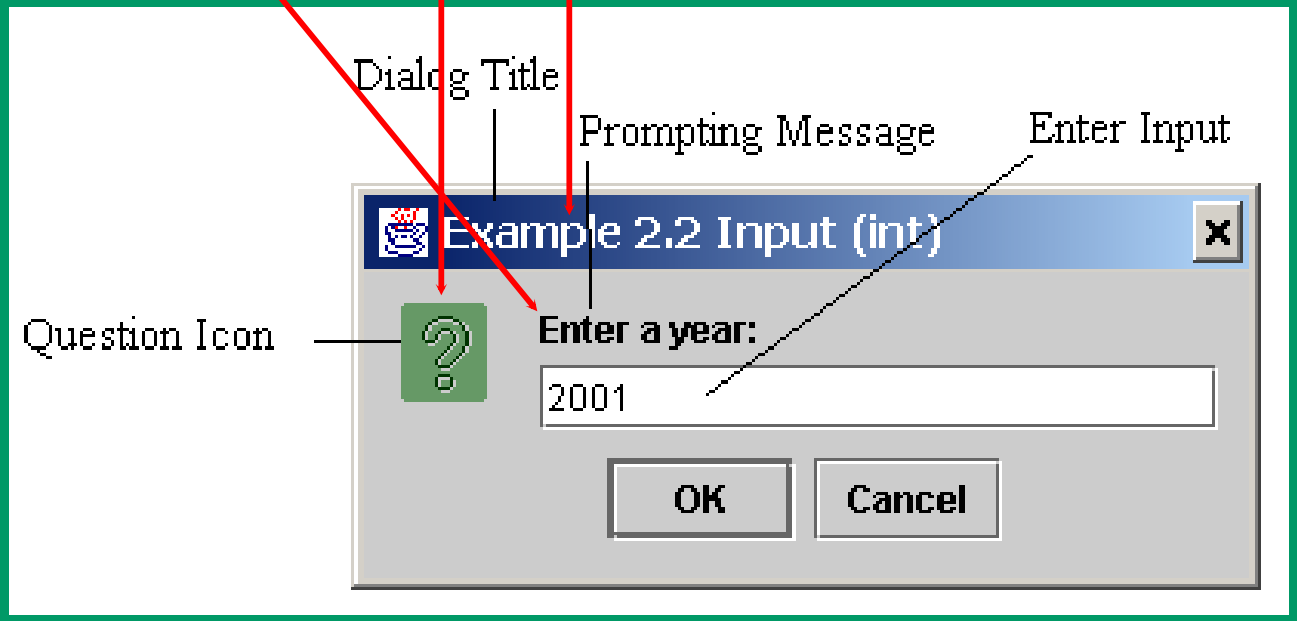
Girdi Diyalog Kutularından Girdi Alma

```
String input =  JOptionPane.showInputDialog(
  "Girdi girin");
```



Girdi Diyalog Kutularından Girdi Alma

```
String string = JOptionPane.showInputDialog(
   null,  "İkna Edici Mesaj",   "Diyalog Başlığı",
   JOptionPane.QUESTION_MESSAGE);
```



Metodu Çağırmanın İki Yolu

showInputDialog yöntemini kullanmanın birkaç yolu vardır. Bunun için şu anda, onu çağırmanın iki yolunu bilmeniz yeterli.

Bunlardan biri, örnekte gösterildiği gibi bir ifade kullanmaktır:

```
String string = JOptionPane.showInputDialog(null, x,
  y, JOptionPane.QUESTION_MESSAGE);
```

x, istem mesajı için bir dize ve y, giriş iletişim kutusunun başlığı için bir dizedir.

Diğeri ise şöyle bir ifade kullanmaktır:

```
JOptionPane.showInputDialog(x);
```

x, istem mesajı için bir dizedir.

Dizgeleri Tam Sayılara Dönüştürme

Giriş iletişim kutusundan dönen değer bir dizgedir. Eğer 123 gibi bir sayısal değer girerseniz, “123” olarak döner. Girişi bir sayı olarak almak için, bir

dizgeyi bir sayıya dönüştürmeniz gerekir.

Bir dizgeyi int değerine dönüştürmek için, Integer sınıfındaki statik parseInt yöntemini aşağıdaki gibi kullanabilirsiniz:

```
int intValue = Integer.parseInt(intString);
```

burada intString “123” gibi bir sayısal dizgedir.

Dizeleri Double'a Dönüştürme

Bir dizeyi double değerine dönüştürmek için, Double sınıfındaki static parseDouble yöntemini aşağıdaki gibi kullanabilirsiniz:

```
double doubleValue =Double.parseDouble(doubleString);
```

doubleString'in "123.45" gibi bir sayısal dize olduğu yer.

Problem: Kredi Ödemelerini Hesaplama

Girdi Diyalogları Kullanarak

kredi ödemelerini hesaplamak için program, girdi giriş diyaloglarından girildi ve çıktı bir çıktı diyalogunda görüntülenir.

$$\frac{krediMiktarı \times faizOranı\ aylıkFaiz}{1 - \frac{1}{(1 + faizOranı\ aylıkFaiz)^{yılSayısı \times 12}}}$$

ComputeLoanUsingInputDialog

Koş